

Exame de Programação Funcional

14 de Julho de 2011

DCC/FCUP

2ª Parte

Nome: _____

Nº mecanográfico: _____

- Esta 2ª parte contém 5 questões e 3 páginas.
- Responda às questões 1 e 2 no espaço marcado no enunciado.
- Responda às questões 3, 4 e 5 numa folha de exame separada.

1. (15%) Escreva uma definição da função *iguais* :: $Eq\ a \Rightarrow [a] \rightarrow Bool$ que verifica se todos os valores duma lista são iguais. Nos casos da lista vazia e com um só elemento o resultado deve ser *True*. Exemplos: *iguais* ['a', 'a', 'a'] = *True*; *iguais* [1, 1, 2, 1] = *False*.

2. (15%) Considere a definição em Haskell dum tipo de dados para árvores binárias de pesquisa:

$$\text{data Arv } a = \text{Vazia} \mid \text{No } a \text{ (Arv } a) \text{ (Arv } a)$$

Complete a seguinte definição da função que insere um valor numa árvore de pesquisa ordenada. Deve manter invariante a propriedade ordenação da árvore e não inserir outra cópia do valor se este já ocorrer na árvore.

```
inserir :: Ord a => a -> Arv a -> Arv a
```

```
inserir x Vazia = _____
```

```
inserir x (No y esq dir)
```

```
    | x < y = _____
```

```
    | x > y = _____
```

```
    | otherwise = _____
```

Responda às questões 3, 4 e 5 numa folha de exame separada.

3. (20%) Um quadrado mágico é uma matriz quadrada de inteiros em que a soma de qualquer linha, coluna e diagonal é uma constante. Por exemplo, a matriz

$$\begin{bmatrix} 6 & 7 & 2 \\ 1 & 5 & 9 \\ 8 & 3 & 4 \end{bmatrix}$$

é um quadrado mágico em que a soma de cada linha, coluna e diagonal é 15.

Escreva uma função *magico* :: $[[Int]] \rightarrow Bool$ que verifica se uma matriz (representada como lista de linhas) é um quadrado mágico. Pode (e deve) decompor a solução usando funções auxiliares; tenha o cuidado de definir todas as funções que necessite e que não façam parte do prelúdio-padrão (ver anexo).

4. (15%) Considere as definições canónicas das funções *take* e *drop* do prelúdio-padrão.

$$take\ 0\ xs = [] \quad (take.0)$$

$$take\ (n + 1)\ [] = [] \quad (take.1)$$

$$take\ (n + 1)\ (x : xs) = x : take\ n\ xs \quad (take.2)$$

$$drop\ 0\ xs = xs \quad (drop.0)$$

$$drop\ (n + 1)\ [] = [] \quad (drop.1)$$

$$drop\ (n + 1)\ (x : xs) = drop\ n\ xs \quad (drop.2)$$

Usando indução sobre n , mostre que

$$take\ m\ (drop\ n\ xs) = drop\ n\ (take\ (n + m)\ xs)$$

para quaisquer inteiros $n, m \geq 0$ e listas xs .

5. (15%) Considere a definição em Haskell dum tipo de dados para árvores binárias de pesquisa.

$$data\ Arv\ a = Vazia \mid No\ a\ (Arv\ a)\ (Arv\ a)$$

- (a) Escreva uma definição recursiva da função *soma* :: $Arv\ Int \rightarrow Int$ que some os valores numa árvore de inteiros.
- (b) Escreva uma definição recursiva da função *foldArv* :: $b \rightarrow (a \rightarrow b \rightarrow b \rightarrow b) \rightarrow Arv\ a \rightarrow b$ tal que *foldArv* $f\ z\ t$ efectua uma operação de *fold* sobre a árvore t , isto é, transforma cada folha na constante z e cada nó numa aplicação da função f .
- (c) Usando indução sobre árvores, mostre que

$$soma\ t = foldArv\ 0\ (\lambda x\ y\ z \rightarrow x + y + z)\ t$$

para qualquer árvore finita t .

Anexo: funções do prelúdio-padrão

Além das funções sobre tipos básicos (e.g. operações aritméticas), pode usar sem definir as seguintes funções sobre listas do prelúdio padrão.

<i>++</i>	<i>!!</i>	<i>head</i>	<i>tail</i>	<i>last</i>	<i>init</i>
<i>take</i>	<i>drop</i>	<i>takeWhile</i>	<i>dropWhile</i>	<i>elem</i>	<i>reverse</i>
<i>length</i>	<i>concat</i>	<i>replicate</i>	<i>repeat</i>	<i>cycle</i>	<i>iterate</i>
<i>sum</i>	<i>product</i>	<i>and</i>	<i>or</i>	<i>all</i>	<i>any</i>
<i>zip</i>	<i>zipWith</i>	<i>map</i>	<i>filter</i>	<i>foldr</i>	<i>foldl</i>